

Report: Reinforcement Learning for Bomberman

Authors:

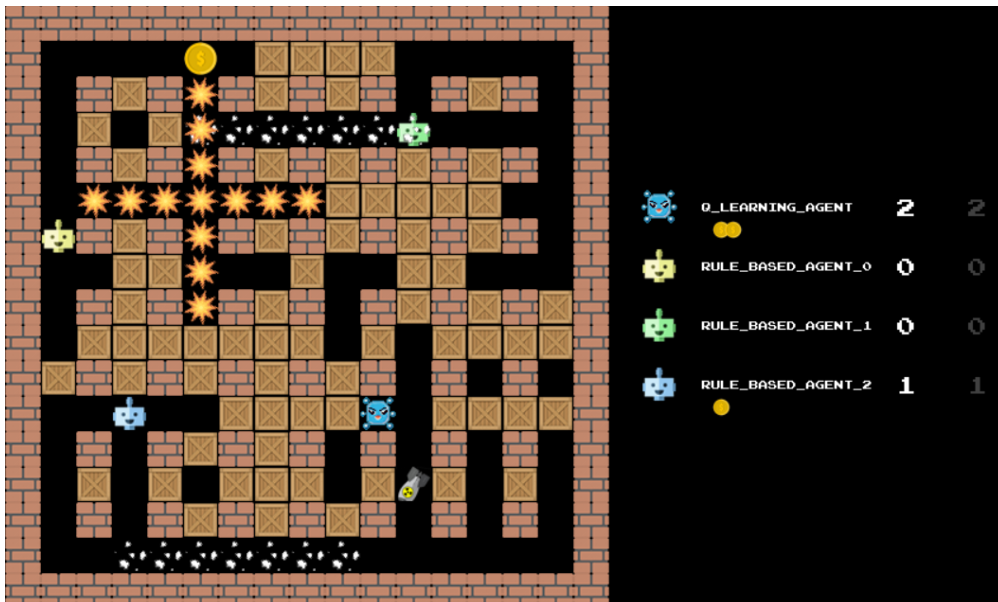
Max Tiedl, 4187550

Bastian Müller, 4138763

Luke Voß, 4084927

GitHub Repository:

https://github.com/lukevoss/Bomberman_RL_2024



A report presented for the course:
Machine Learning Essentials

Ruprecht-Karls-University Heidelberg
Faculty of Mathematics and Computer Science
Department of Computer Science
16.08.2024

Abstract

This report explores the application of Reinforcement Learning techniques to develop competitive agents for the classic game Bomberman. Specifically, we implement and compare two major RL algorithms: Q-learning, a value-based approach, and Proximal Policy Optimization (PPO), a policy gradient method. The goal is to investigate the effectiveness of these algorithms in a multi-agent, adversarial environment with complex decision-making scenarios. The report details the design of both agents, covering their respective feature extraction processes, reward structures, and training methodologies. Through extensive experiments and evaluation, we analyze the performance of each agent, highlighting their strengths, weaknesses, and potential improvements. The results demonstrate that while both agents can successfully navigate the challenges of Bomberman, the PPO agent outperforms Q-learning in terms of strategic decision-making and adaptability in a multi-agent environment. Finally, we discuss the challenges faced during implementation, such as balancing exploration and exploitation, managing large state spaces, and optimizing safety in high-risk scenarios. Future work will focus on refining agent strategies and exploring hybrid methods to improve performance further.

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Goals and Challenges	3
1.2.1	Goals	3
1.2.2	Problem setting	3
1.2.3	Challenges	4
1.3	Report structure	4
2	Background	5
2.1	Reinforcement Learning	5
2.1.1	Model-based vs. Model free	6
2.1.2	On-Policy vs Off-Policy	6
2.1.3	Value-Based vs. Policy based	7
2.2	Q-learning	7
2.2.1	Algorithm	8
2.2.2	Exploration vs Exploitation	9
2.2.3	Strengths and Weaknesses	10
2.3	Proximal Policy Optimization	11
2.3.1	Algorithm	11
2.3.2	Training Loop	13
2.3.3	Strengths and Weaknesses	13
2.4	The Bomberman Game	14
2.4.1	Overview	14
2.4.2	Rules	14
2.4.3	Known Strategies	15
2.4.4	Game Scenarios	15
2.4.5	Game Research	15
2.4.6	Game Applications	16
3	Project planning	17

4	Methods	19
4.1	Feature design	19
4.1.1	Simple feature extraction	19
4.1.2	Complex feature extraction	22
4.2	Q-learning agent	23
4.3	Proximal Policy agent	24
4.3.1	Actor-Critic	24
4.3.2	Network Architecture	25
4.3.3	Feature Extraction and Action Selection	25
4.3.4	Training Process	26
4.3.5	Mini-Batch Training and Optimization	26
4.4	Imitation learning	27
4.4.1	Expert Dataset	27
4.4.2	Training Procedure	28
4.5	Evaluation metrics	28
4.5.1	Existing metrics	28
4.5.2	Custom metrics	28
5	Training	31
5.1	Q-learning Agent	31
5.1.1	Reward structure	31
5.1.2	Pre-training	33
5.1.3	Main training	34
5.2	Proximal Policy Agent	35
5.2.1	Imitation Training	35
5.2.2	Main training	35
5.2.3	Hyperparameter Tuning	36
6	Experiments and Results	37
6.1	Training results	37
6.1.1	Training process of the Q-learning agent	37
6.1.2	Results for the PPO agent training	39
6.2	Performance comparison	41
6.2.1	Q-learning agent	41
6.2.2	PPO agent	43
6.2.3	PPO vs. Q-learning	43
7	Conclusion and Outlook	46
7.1	Conclusion	46
7.2	Outlook	46
7.2.1	Future Work	46
7.2.2	Improvements	47

Acronyms

AI Artificial Intelligence

BFS Breadth-first search

CNN Convolutional Neural Network

GAE Generalized Advantage Estimation

GRU Gated recurrent unit

LSTM Long Short-Term Memory

MCTS Monte Carlo Tree Search

MDP Markov Decision Process

ML Machine Learning

MLP Multi-Layer Perceptron

MSE Mean Squared Error

PPO Proximal Policy Optimization

ReLU Rectified Linear Unit

RL Reinforcement Learning

SARSA State-Action-Reward-State-Action

TD Temporal Difference

TRPO Trust Region Policy Optimization

Chapter 1

Introduction

BASTIAN MÜLLER

The following two sections aim to motivate this report as well as outline our goals and challenges. They will illustrate, why this topic is relevant and highlight several approaches to solve the stated problem. The third and last section will discuss the structure of this report in more detail.

1.1 Motivation

As one of the oldest fields in the Machine Learning (ML) space, coming up in the 1950s, Reinforcement Learning (RL) gained international media coverage since the victory of AlphaGo over Professional GO player Lee Sedol in March 2015. The success of AlphaGo was a milestone in the field of Artificial Intelligence (AI) and ML and has shown the potential of RL in solving complex problems. Even though RL based approaches have been successful in playing games like Backgammon professionally (Sutton and Barto 2020, Ch. 11.1) already in 1992 or almost all retro Atari games in 2013 (Mnih et al. 2013), none of these achievements tackled a game as complex as the game of GO.

RL is a subfield of ML that deals with the issue of learning how to act optimally in an environment. Géron 2019 brings the major components together in a single sentence on page 610:

“In Reinforcement Learning, a software *agent* makes *observations* and takes *actions* [based on a *policy*] within an *environment*, and in return it receives *rewards*.”

The agent is the entity that interacts with the environment. The environment is the world in which the agent operates. Based on the outcome of the agents actions it receives positive or negative rewards. The overall goal of the agent is to maximize the cumulative reward over time.

The problem we face is similar to the ones DeepMind targeted in 2013. The classic arcade game Bomberman was released in Japan in 1983 where it was developed by

video game company Hudson Soft. The game features a multiagent environment with complex decision-making and strategy requirements. This is similar to various real-world problems, which involve multiple agents, obstacle navigation, adversarial competition, dynamic environments, risk and reward. Thus, developing a competitive Bomberman agent presents a complex, interesting, and relevant challenge.

1.2 Goals and Challenges

This section covers our goals, goes shortly over the problem setting as well as certain challenges we face.

1.2.1 Goals

The primary goal of this report is to display our understanding of RL and its application in the context of training a competitive Bomberman agent. Using methods from RL, the agent will learn to navigate the game environment, place bombs strategically, and optimize its actions to maximize its overall score. To showcase our understanding, we not only apply the techniques but also explain several of them with specific focus on those we chose to solve the stated problem. Furthermore, we describe our testing process and share insights into our training. Towards the end of the report, we shift our focus to critique our own approach and suggest improvements.

As for the development of our agent, our primary goal is to beat the rule-based-agents provided consistently. After observing the rule-based agents and trying to beat them manually, we realized that coming up with an agent that can beat three of them consistently is a challenging first task.

From talks with students who participated in the competition last year, we learned that we had to clear at least 5 points on average per round against three rule-based agents to have a chance to win. So, naturally, we set this as our next milestone.

1.2.2 Problem setting

Bomberman is a grid-based, multiagent game in which four agents compete in discrete time steps. The game is played over 400 rounds with four agents initially. During each time step (round), an agent can move up, down, left, or right, drop a bomb, or remain still. Movement is restricted to empty tiles, as walls, crates, bombs, and other agents block the agent's path. Each agent earns points by collecting coins, revealed by blowing up crates, and by eliminating other agents by blowing them up. The agent must learn to place bombs strategically and avoid being caught by their own bombs. The random placement of crates, coins and the initial agents position introduce the need for adaptability. The overall goal for the agent is to score as many points as possible by the end of the game. The end of the game is reached when either no agents are left to play or after 400 rounds.

1.2.3 Challenges

Designing a RL agent presents several challenges. First, the agent must learn to balance different objectives, such as collecting coins, destroying crates, and eliminating opponents, while simultaneously ensuring its own survival. The game environment includes both static and dynamic obstacles, such as walls and bombs, which the agent must learn to navigate. Deciding for an action that is impossible might kill the agent itself, which is why we paid significant attention to avoiding illegal actions. Additionally, the unpredictability of opponent behavior introduces further complexity, requiring the agent to develop strategies that are both adaptive and robust. Time constraints during decision-making (0.5 seconds per step) add further difficulty, as the agent must make quick and effective choices. Given the time constraints, we chose to go with simpler and faster methods instead of employing state-of-the-art models such as the Transformers' architecture (Vaswani et al. 2023). Finally, the randomization of the game elements (such as crate and coin placement) and the requirement to perform well across multiple rounds and episodes demand a well-generalized strategy that can handle different game configurations.

1.3 Report structure

As requested, the report starts with an introduction in Chapter 1 that provides an explanation of the problem, highlights its relevance, and outlines the goals and challenges of the work. Following the introduction, the report proceeds to explain the methods used in the project in Chapter 2. In Chapter 3 we describe the organization of our team, as well as, how and what we worked on during this project. While Chapter 4 focuses on how we adjusted the methods introduced in Chapter 2 to fit our needs, Chapter 5 describes the application of these methods and our approach to training the agent. The results of our work can be read in Chapter 6, while we summarize and critique our own work in the last Chapter 7.

Chapter 2

Background

This chapter introduces RL, outlining its various model categories (see Section 2.1) and detailing the core principles of the algorithms implemented in this project. Section 2.2 introduces the basic forms of RL where the state and action spaces are sufficiently small, allowing the action-value function to be represented as a table, commonly known in RL as Q-learning. In such scenarios, these methods are often capable of finding exact solutions, meaning they can precisely determine the optimal value function and policy (Sutton and Barto 2020). In contrast, we explore approximate methods, such as described in Section 2.3. While these methods only yield approximate solutions, they are adaptable to much larger problems, offering effective solutions where exact methods are impractical. Lastly, we provide an introduction to the Bomberman Game, which serves as the foundational task for this project (see Section 2.4).

2.1 Reinforcement Learning

LUKE VOSS

Reinforcement Learning is a distinctive branch of machine learning that focuses on how agents take actions in an environment to maximize some notion of cumulative reward. RL differs from other types of machine learning in that it is primarily concerned with making decisions sequentially and learning from the outcomes of these decisions, rather than from static datasets. In essence, a RL agent learns to map situations to actions in a way that maximizes a long-term reward signal (Sutton and Barto 2020). RL has been successfully applied to a range of tasks from simple problems, like navigating mazes, to complex challenges like playing video games at a superhuman level (Silver et al. 2017), autonomous driving (Sallab et al. 2017), and recently to training large language models (Bai et al. 2022). This field continues to grow, driven by advances in algorithmic design, computational resources, and the increasing availability of large and complex datasets.

The basic framework of RL involves three key elements: an agent, an environment, and a reward function. The agent interacts with the environment, which is typically formulated as a Markov Decision Process (MDP), by choosing actions based on a policy.

In response to each action, the environment provides the next state and an associated reward. The goal of the agent is to learn a policy that maximizes the total accumulated reward over time, often discounted by a factor that makes future rewards worth less than immediate rewards (Sutton and Barto 2020). This interactive process is denoted in Figure 2.1.

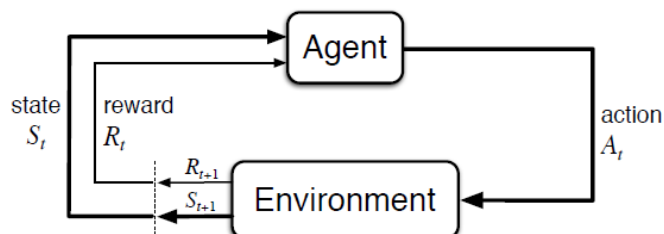


Figure 2.1: The agent-environment interaction in RL. Figure taken from Sutton and Barto 2020.

RL techniques are broadly categorized based on their approach to learning and decision-making strategies. Below, we outline the distinctions between these categories

2.1.1 Model-based vs. Model free

Model-based RL algorithms involve the agent building a model of the environment based on its interactions. This model represents the dynamics of the environment, predicting future states and rewards for actions taken. Such predictions allow the agent to plan by considering potential future situations before they occur, which can lead to more efficient learning and decision-making. Model-based methods are particularly useful in environments where constructing an accurate model is feasible and where long-term planning is crucial (Moerland et al. 2023).

In contrast, model-free methods do not attempt to construct a model of the environment. Instead, they focus directly on learning the best action to take in each state, based on the rewards received. These methods typically require more interactions with the environment to learn effective strategies, as they operate without the foresight provided by a model. However, they are simpler and more versatile, making them suitable for a wide range of applications where building an accurate model of the environment is challenging or unnecessary (Moerland et al. 2023).

2.1.2 On-Policy vs Off-Policy

On-policy methods learn the value of the policy being used by the agent, meaning the learning process evaluates and improves the same policy that determines the agent's actions. A key example of an on-policy method is State-Action-Reward-State-Action (SARSA) (Rummery and Niranjan 1994), where the policy learned is directly influenced by the actions taken by the agent. This approach ensures that the policy evaluation is consistent with the policy execution, promoting stable and consistent learning.

Off-policy methods, on the other hand, separate the policy used for making decisions (behavior policy) from the policy being improved (target policy). This allows the agent to learn from actions that are outside its current strategy, providing a broader view of possible strategies. A prominent example of off-policy learning is Q-learning (C. Watkins 1989), where the learning phase involves updating a policy that is not necessarily executed during learning. Off-policy methods can be more efficient in exploring and learning from a diverse set of experiences (Sutton and Barto 2020).

2.1.3 Value-Based vs. Policy based

Value-based focuses on determining the value of each state or state-action pair in an environment. The policy is derived indirectly from these values—typically by choosing the action that maximizes the value function in each state. These methods excel in environments where the value functions can be accurately estimated, and the policy can be easily derived from these estimates (Sutton and Barto 2020).

Policy-based methods directly adjust the parameters of the policy based on the performance of the agent. These approaches are particularly beneficial in high-dimensional or continuous action spaces where value-based approaches might struggle. Policy-based methods often utilize gradient ascent techniques to directly optimize the policy, improving the expected return. One of the advantages of policy-based is the ability to learn stochastic policies, which provide a probabilistic distribution over actions, enabling exploration and handling of varying degrees of uncertainty in the environment (Grondman et al. 2012).

2.2 Q-learning

LUKE VOSS

Q-learning is a model-free, value-based, off-policy RL algorithm known for its efficacy in finding optimal action sequences within Markovian domains. This algorithm empowers agents to learn how to act optimally through direct interactions with the environment, rather than requiring a predefined model of the world. Q-learning enables an agent to evaluate the potential of its actions based on both immediate outcomes and anticipated future rewards, all without needing to follow a fixed policy (C. J. C. H. Watkins and Dayan 1992).

The foundational mechanism of Q-learning is rooted in the concept of Temporal Difference (TD), as originally outlined by (Sutton 1988). This method involves the agent making decisions in a given state, executing actions, and observing the outcomes in terms of immediate rewards or penalties and the estimated values of resultant states. This learning process is iterative, where the agent repeatedly tests all possible actions in all conceivable states. Through this extensive exploration, the agent gradually recognizes the actions that yield the highest long-term discounted rewards, thereby crafting a strategy that is optimized over time.

In this section, we will look at the core components of the Q-learning algorithm: the training process utilizing the Bellman Equation and the Q-table for learning state-action values, the implementation of epsilon-greedy exploration to balance the trade-off between exploration and exploitation, and a thorough analysis of the strengths and weaknesses of Q-learning.

2.2.1 Algorithm

The centerpiece of Q-learning is the Q-table, a matrix where each row represents a state and each column represents an action. The entries in this table estimate the total amount of reward an agent can expect to accumulate over the future, starting from that state and taking that action. As illustrated in the flowchart 2.2, the Q-learning process involves several key steps.

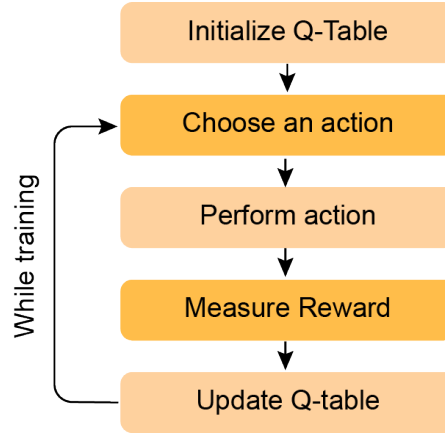


Figure 2.2: Updating of Q-table during training.

Initially, the Q-table is filled with zeroes or some other small random values. This represents the agent's lack of knowledge before it starts interacting with the environment. At each state, the agent selects an action. This selection can be random but is most commonly performed using a policy like epsilon-greedy, which balances exploration and exploitation (see Section 2.2.2). The agent performs the selected action and observes the reward. This reward is specific to the action and the state from which it was taken. Finally, the Q-table is iteratively updated using the Bellman Equation 2.1.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right) \quad (2.1)$$

where:

- $Q(s, a)$ is the current Q-value for state s and action a .
- α is the learning rate, which determines how much new information affects the existing Q-value.

- r is the reward received for taking action a in state s .
- γ is the discount factor, used to balance the importance of immediate and future rewards.
- $\max_{a'} Q(s', a')$ represents the optimal future value from the new state s' .

The Bellman Equation provides a recursive update to the Q-values and is fundamental to the convergence of the Q-learning algorithm. It ensures that the Q-values converge to the optimal values, which represent the maximum cumulative reward that can be obtained starting from any given state, taking any action, and thereafter following the best possible policy (C. Watkins 1989). Through repeated interactions with the environment, and by following the update rules as defined by the Bellman Equation, the Q-learning algorithm refines the Q-values within the table, improving the policy followed by the agent. This ongoing learning process allows the agent to make increasingly better decisions, maximizing the expected cumulative reward over time. The Q-table thus serves as an essential component of the strategy, directing the agent's actions based on its learned experiences (C. J. C. H. Watkins and Dayan 1992).

2.2.2 Exploration vs Exploitation

A fundamental challenge in RL is balancing exploration and exploitation. This dilemma can be understood as the choice between exploring the environment to find new knowledge and exploiting known information to maximize rewards. Q-learning addresses this challenge primarily through the epsilon-greedy strategy during the training phase (C. J. C. H. Watkins and Dayan 1992).

The epsilon-greedy algorithm is a simple yet effective way to balance exploration and exploitation. Its policy is denoted as:

$$a = \begin{cases} \text{random action} & \text{with probability } \epsilon, \\ \arg \max_{a'} Q(s, a') & \text{with probability } 1 - \epsilon. \end{cases} \quad (2.2)$$

With probability ϵ , the agent selects an action at random, facilitating exploration of the environment. This randomness allows the agent to discover and learn about the rewards associated with new actions and states that it might not otherwise select during the exploitation phase. The action that has the highest estimated reward according to the current Q-table is chosen with a probability of $1 - \epsilon$, promoting exploitation. This step ensures that the agent typically selects actions that are known to yield high rewards, based on previous learning. The parameter ϵ is crucial and typically starts high (encouraging more exploration) and decays over time as the agent learns more about the environment, allowing for more exploitation of the known good actions as the agent becomes more confident in its knowledge (Sutton and Barto 2020). A comparison of different fixed ϵ -parameter is shown in Figure 2.3

During the evaluation phase, where the goal is to test the effectiveness of the learned policy, the agent predominantly exploits its learned knowledge. This is achieved by

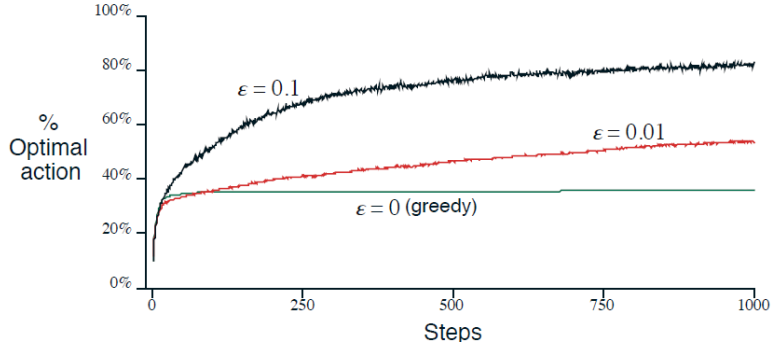


Figure 2.3: Average performance of ϵ -greedy action-value methods on a 10-armed bandit testbed. These data are averages over 2000 tasks. All methods used sample averages as their action-value estimates. Caption and figure taken from (Sutton and Barto 2020).

choosing actions solely based on the highest Q-value in the current state, using the policy:

$$\pi(s) = \arg \max_a Q(s, a)$$

Here, $\arg \max$ selects the action at state s that has the highest Q-value from the Q-table, reflecting the best-known action to take according to the accumulated learning.

The epsilon-greedy approach provides a simple and practical framework for handling the exploration-exploitation trade-off. It ensures that the learning process does not become trapped in a local optimum early on, and it helps explore potentially better options that may not have been obvious from initial observations. Over time, as the agent shifts towards exploitation, the accumulated knowledge captured in the Q-table directs the agent towards the most rewarding actions, maximizing the effectiveness of the decision-making process in the learned environment (Sutton and Barto 2020).

2.2.3 Strengths and Weaknesses

Q-learning, as a prominent model in the field of RL, presents both significant advantages and certain limitations. Its strengths include its model-free nature, which allows it to operate without prior knowledge of the environment's dynamics, making it adaptable to a wide array of situations. Furthermore, as an off-policy learner, Q-learning is capable of learning the optimal policy regardless of the agent's action choices, effectively learning from exploratory actions that may not be part of the current strategy. This characteristic is strengthened by its proven convergence properties, given that all state-action pairs are visited infinitely often and the learning rate is appropriately set. Additionally, Q-learning is valued for its simplicity and versatility, offering straightforward implementation for discrete tasks and broad applicability across diverse domains (Beysolow II 2019).

However, Q-learning is not without its challenges. The convergence speed can be slow in environments with large state or action spaces, as the algorithm requires extensive updates based on observed rewards and next-state values. The maintenance and updating

of a Q-table, which Q-learning relies on, can be both memory and computationally intensive, particularly as the size of the state and action space expands, making it incapable for tasks with larger more complex states. Furthermore, a significant limitation arises when the agent encounters states that have not been previously seen. In such cases, the algorithm lacks prior data to guide decisions, necessitating an essentially random choice. This limitation underscores the critical need for the agent to explore every possible state during the training phase to enhance its effectiveness, consequently making it unsuitable for tasks where such comprehensive exploration is impractical or impossible. Lastly, its design for Markovian domains means that its effectiveness decreases in environments where the Markov property does not hold, as predictability based solely on the current state and action becomes less reliable (Beysolow II 2019).

In conclusion, although Q-learning provides robust tools for navigating complex environments, its success depends on carefully addressing its limitations and tailoring its application to the specific conditions of the domain.

2.3 Proximal Policy Optimization

MAX TIEDL

Proximal Policy Optimization (PPO) is a popular RL algorithm that falls under the category of policy-based, model-free methods. Unlike value-based methods such as Q-learning, which rely on estimating the value of states or state-action pairs to derive an optimal policy, PPO directly optimizes the policy that determines the actions an agent should take in each state. It uses two neural networks, one for the policy (actor) and one for the value function (critic), to estimate how good actions are and to update the policy.

Policy-based methods are often preferred in continuous action spaces, where value-based methods struggle. PPO is known for its efficiency and stability, making it one of the most widely used policy gradient methods. It is often preferred over Trust Region Policy Optimization (TRPO) because it is easier to implement and has fewer computational complexities,

In contrast to value-based methods like Q-learning, policy-based methods work by adjusting the policy’s parameters to maximize a predefined objective.

2.3.1 Algorithm

Recap over previous methods

One common approach to this optimization problem is the policy gradient method, which updates the policy by ascending the gradient of the expected return. However, policy gradient suffers from several limitations, including high variance in gradient estimates and instability due to large policy updates. To address these issues, algorithms like TRPO were developed, which constrain the step size of updates to avoid drastic changes in the policy.

The core of TRPO is the “surrogate” objective function which is maximized subject to a constraint on the KL divergence between the old policy $\pi_{\theta_{old}}$ and the new policy π_{θ} (Schulman, Wolski, et al. 2017).

$$L(\theta) = \hat{\mathbb{E}}_t \left[r_t(\theta) \hat{A}_t \right] = \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \hat{A}_t \right] \quad (2.3)$$

- $\pi_{\theta}(a_t|s_t)$ is the probability of taking action a_t in state s_t under the current policy π_{θ}
- $\pi_{\theta_{old}}(a_t|s_t)$ is the probability under the old policy
- $\hat{A}_t$ is the estimated advantage function at time step t
- $\hat{\mathbb{E}}_t$ is the empirical average over a finite batch of samples

This constraint ensures that each update to the policy doesn’t change it too drastically, preventing large updates that could destabilize learning. The idea is to keep the policy changes small so that the learning process remains stable.

TRPO solves this problem approximately using a combination of the conjugate gradient algorithm (for efficient optimization) and a quadratic approximation for the constraint.

New objective

While TRPO is effective, the need to solve a constrained optimization problem with KL-divergence and the conjugate gradient method adds complexity and computational overhead. To simplify TRPO while retaining stability, the paper by (Schulman, Wolski, et al. 2017) introduces a modification to the surrogate objective.

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (2.4)$$

The added term modifies the previous equation by clipping the probability ratio, thus ensures r_t to be inside the interval $[1 - \epsilon, 1 + \epsilon]$. The idea behind clipping is to prevent the policy update from being too large, ensuring that the ratio between the new and old policy stays within a small, predefined range. This makes PPO more stable compared to other policy gradient methods.

ϵ is a hyperparameter, which is set to 0.2 in our implementation. A low value for ϵ can lead to insufficient updates, while a too large value can cause instability by allowing large updates.

By using the minimum between the original and clipped objectives, PPO discourages changes that would lead to high variance in policy updates. The advantage function $\hat{A}_t$ is often calculated with the Generalized Advantage Estimation (GAE) method (Schulman, Moritz, et al. 2018). This method is often used to reduce the variance in the advantage estimates, while still maintaining an unbiased estimate of the gradient.

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \quad (2.5)$$

- γ is the discount factor. It is used to reduce the variance by downweighting rewards corresponding to delayed effects, at the cost of introducing bias.
- λ controls the trade-off between bias and variance in the advantage estimates, with a higher value reducing the variance.

The last parameter is the TD residual:

$$\delta_t = r_t + \gamma V(s_{t+1}) - v(s_t) \quad (2.6)$$

This can be considered as an estimate of the advantage of the action a_t .

2.3.2 Training Loop

PPO, like other policy gradient methods, operates in an iterative loop where the agent interacts with the environment to gather experience, and then uses this data to improve the policy.

1. **Data Collection:** The agent collects experience by interacting with the environment using the current policy. For each time step, the agent observes a state s_t , selects an action a_t based on the policy, and receives a reward r_t . This experience is stored in a buffer for further processing.
2. **Advantage Estimation:** PPO uses the advantage function $\hat{A}_t$ to guide policy updates. The advantage function is the difference between the observed return and the expected return under the current policy.
3. **Policy Update:** After gathering enough data, PPO performs several epochs of updates to the policy parameters using the collected experience. The policy is updated by maximizing the clipped objective function $L^{CLIP}(\theta)$.
4. **Repeat:** The policy is updated iteratively, with new data collected in each iteration. This process continues until the policy converges or the performance stops improving.

2.3.3 Strengths and Weaknesses

PPO's clipped objective function ensures that policy updates are not too large, making the training process more stable compared to standard policy gradient methods or even more complex approaches like TRPO. By limiting the extent to which the policy can change between updates, PPO avoids the risk of drastic policy shifts that can lead to instability or performance collapse. Another advantage is that unlike methods such as

Q-learning, which often require large batches of samples to compute Q-values, PPO can work effectively with smaller batches of data due to its on-policy nature and frequent updates. PPO also works well across a range of environments, from simple problems like this Bomberman game to more complex tasks involving continuous action spaces, such as robotics or complex video games. Its ability to handle both discrete and continuous action spaces makes it highly flexible. Also, an important point for the practical implementation is that it works good with parallelization, which is one reason it's commonly used in large-scale RL problems.

On the other hand the on-policy nature can lead to an inefficient training process because the agent has to interact with the environment to generate new samples, but this applies primarily for environments where data collection is expensive. Off-policy methods use past experiences, which makes them more sample efficient. A lot of time is also needed for tuning the different hyperparameters such as the clip range ϵ or the learning rate. Poor choices of hyperparameters can result in suboptimal performance.

2.4 The Bomberman Game

BASTIAN MÜLLER

2.4.1 Overview

Our version of the previously introduced game Bomberman takes place in a 15x15 grid which is surrounded by walls. In the grid, every second row and column has alternating walls and crates or an empty space. By default, the crate density is set to 75% and the amount of coins to 9, while the probability of getting a coin from a crate is 10% as calculated below.

$$\begin{aligned}
 \mathbb{P}(\text{Coin in crate}) &= \frac{\text{Coins}}{\text{Amount of crates}} \\
 &= \frac{\text{Coins}}{\text{Density} \cdot \text{Potential spaces}} \\
 &= \frac{9}{0.75 \cdot (6 * 7 + 13 * 6)} \\
 &= 0.1
 \end{aligned} \tag{2.7}$$

2.4.2 Rules

During each time step (round), an agent can choose from a set of moves.

- Move up
- Move down
- Move left
- Move right

- Drop bomb (if possible)
- Remain still

If an agent moves on a field that is blocked by a wall, bomb, enemy or crate, the action is illegal, and the agent will instead do nothing. If an agent places a bomb and tries to place another bomb in the next four steps, it is also illegal, as the bomb is not available yet. Collecting a coin yields one point, while killing an enemy with a bomb yields 5 points by default. The game ends either after 400 rounds are played or until no agent is alive¹. The player with the highest score wins the round.

2.4.3 Known Strategies

Most documented strategies focus on more complex versions of the game. For example, this article² suggests that the player is better off using his weak bomb in the beginning to trap enemies with their bombs before the enemy discovers the power-up. Since our, much simpler, version of the game has neither power ups nor different bomb strengths, it is difficult to extract valid strategies from the articles found online. Another article³ just reiterates the importance of dropping the bombs strategically and carefully but also mentions that the player needs to pay attention to power-ups or other modifications to the map. Except for (Kowalczyk et al. 2022), which is discussed later, these articles and others like them were not helpful in finding a strategy for our Bomberman agent.

2.4.4 Game Scenarios

Next to the default scenario (or classic scenario), we used multiple other environments for training our agents, which put the focus on different key aspects of the game. In addition to the scenarios provided to us, we defined several additional ones and summarized them in Table 2.1.

The first scenario simulates that paths are shown for a coin and possibly for an opponent. The second scenario shows paths for crates and possibly opponents from the starting point. The third scenario shows paths for all three goals: opponents, coins, and crates from the starting point. In the default version of the game it takes some time until the path to other players is cleared or coins are lying around openly. Overall, these scenarios are designed to help the agent learn the game faster and more efficiently with a specific focus on rarer events.

2.4.5 Game Research

While there exists loads of research on RL, research on Bomberman specifically is rare. The only paper we could find that is directly targeting the creation of a Bomberman

¹During training, the game ends by default when the trained agent dies.

²<https://strategywiki.org/wiki/Bomberman/Walkthrough>, Accessed: 22.09.2024

³<https://www.gameskinny.com/tips/super-bomberman-r-beginner-tips-and-tricks/>, Accessed: 22.09.2024

Table 2.1: Custom scenarios

Origin	Density	#Coins	$\mathbb{P}(\text{Coin in Crate})$	Description
Provided	0	0	0	Movement
	0	9	0	Collect coins
	0.75	50	0.56	Loot crates
	0.75	9	0.1	Classic game
Ours	0	1	0	Movement
	0.02	0	0	Crates only
	0.02	9	1	Classic game

agent is (Kowalczyk et al. 2022). The six-person team of the University of Wrocław studied approaches to play a Bomberman variant on the CodinGame platform. Among there approaches were Beam Search, Monte Carlo Tree Search (MCTS) and Rolling Horizon Evolutionary Algorithms.

Other papers like (Meisheri et al. 2019) focus on the Bomberman clone “Pommerman” (Resnick et al. 2022) that was developed as a benchmark for AI agents which also offers multiple modes. The best results were achieved by researchers utilizing the MCTS algorithm.

2.4.6 Game Applications

Studying RL methods in any dynamic environments with uncertainty and unpredictability of others is worthwhile and relevant because the same methods we use to develop a successful Bomberman agent are used - although involving much more complex methods - in real world applications like recommendations, trading, robotics or automation. While the insights gained from studying this game in particular may be limited, the field of RL has countless practical applications.

Chapter 3

Project planning

LUKE VOSS

At the beginning of our final project, we held a kick-off meeting to outline our respective timeframes and to strategize for the upcoming challenges. Given our conflicting schedules, we determined that simultaneous collaboration would be mostly impossible. Consequently, we resolved to work predominantly independently, leveraging the prior research conducted by our team.

This approach posed significant collaborative challenges, prompting us to devise strategies to mitigate these issues. To maintain cohesion and facilitate a continuous exchange of ideas, we established weekly progress meetings. These sessions were designed to keep team members up-to-date of recent developments, share experimental results, and promote active discussions on potential solutions to the problems being addressed.

During our initial meeting, we identified Q-learning as a promising approach, owing to its model-free nature which allows for flexible experimentation with various reward structures and feature extractions.

Luke Voß initialized the project by developing a basic Q-learning agent to serve as a foundation for more advanced models. He experimented with numerous reward systems and feature extractions, ultimately achieving notable success. His implementation resulted in an average score of 5.1 per game, significantly surpassing the performance of rule-based agents. Although he explored the application of imitation learning, it proved to be ineffective for Q-learning.

Subsequently, Luke also integrated PPO and imitation learning within the PPO framework. However, while imitation learning showed potential, being able to quite efficiently mimic the rule-based agent, the performance of PPO algorithm was still bad due to an inappropriate choice of hyperparameters. From that point on, Max Tiedl took the lead, exploring more sophisticated methods and succeeding in implementing a highly effective PPO agent that managed to surpass our Q-Learning agent.

With two effective strategies in place, Bastian Müller directed his efforts toward developing a comprehensive logging and evaluation framework. He enhanced the detail

of logged events, including the agents' locations and additional metadata, and later parsed these logs using regular expressions. From this data, he recalculated the training processes and produced various graphs and tables for our report.

In the last week before the submission deadline, we convened for an extensive meeting and rigorously tested all our agents to identify the most efficient one for our final submission.

Maintaining close contact throughout the project, despite our differing schedules, was crucial to the success of our project. It prevented code duplication, fostered a structured and organized coding environment, and ensured cohesive project development through to its conclusion and subsequent report

Chapter 4

Methods

In this chapter, we detail the methodologies employed to train our agents using Q-learning and PPO. We begin with the design and rationale behind the feature vectors used to interpret and navigate the game environment efficiently (see Section 4.1). This includes both simplistic approaches conducive to Q-learning and more complex strategies that leverage the capabilities of PPO. Furthermore, we discuss the specific configurations, adjustments, and the overall framework of the training regimes that allow these agents to perform effectively in their respective settings. Detailed descriptions of our Q-learning and PPO agents are provided in Sections 4.2 and 4.3, respectively. In Section 4.4, we introduce our approach to imitation learning, which has significantly enhanced our PPO agent training. Finally, Section 4.5 details the evaluation metrics we use to assess the performance and efficacy of our agents.

4.1 Feature design

To craft a feature vector suitable for low-power computational devices, we integrated multiple strategies, drawing substantial inspiration from existing research, notably Kowalczyk et al. 2022. Initially, our approach concentrated on extracting the critical information into a concise, straightforward feature vector. This simplified design is essential for the application of linear methods like Q-learning, as outlined in Section 4.1.1. We applied this architecture in our baseline Q-learning agent. Subsequently, we explored more complex feature extraction methods enabled by the PPO algorithm, which are elaborated upon in Section 4.1.2.

4.1.1 Simple feature extraction

LUKE VOSS

For our initial feature design, we focused on extensive preprocessing of the game state to recommend specific actions aligned with various goals, thereby facilitating the development of both the agent and its reward structure on low computational power machines.

Insights from Kowalczyk et al. 2022 were key in adapting these strategies to our specific environment. This strategy enabled rapid testing and baseline model establishment. Additionally, making each game state human-readable enhanced logging capabilities: at any state, we could document the action suggestions for the agent and assess its learning progress. Our baseline feature vector design is illustrated in Figure 4.1.

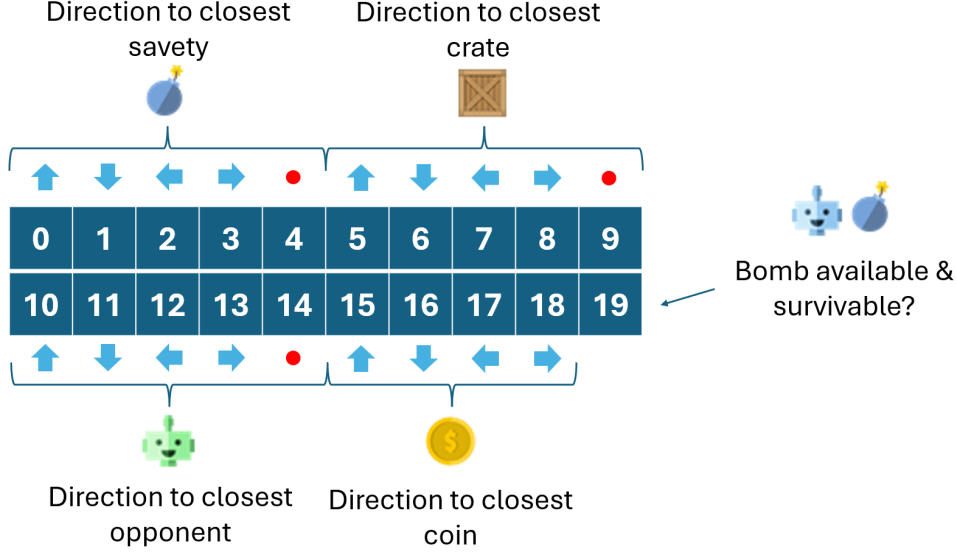


Figure 4.1: Simple feature vector design: Directions to four specific goals are calculated using a breadth-first search algorithm (Section 4.1.1), with the results encoded into one-hot vectors. Each vector reserves the first four positions to denote the directions UP, DOWN, LEFT, and RIGHT. The role of an optional fifth position varies, indicating either WAIT or BOMB, depending on the relevant goal. The final position in the vector indicates whether a bomb is available and whether it can be safely deployed. Inspiration was drawn from Kowalczyk et al. 2022.

This design includes action suggestions for four distinct goals: the next safe tile, the next crate, the next opponent, and the next coin, along with a boolean value on whether placing a bomb is viable and survivable, creating a vector of length 20. The shortest paths are determined using a breadth-first search algorithm, as detailed in Section 4.1.1. For each goal, the direction vector is one-hot encoded to represent five potential actions (UP, DOWN, LEFT, RIGHT, WAIT/BOMB) and a 'No found path' state, except for the coin direction which excludes the WAIT/BOMB state. The simplification to a single action suggestion per goal significantly reduces the dimensionality of the search space from 2^{20} to $6 \times 6 \times 6 \times 5 \times 2 = 2160$. Each of the first five positions in our vector indicates the direction to the nearest safe tile, with the fifth position indicating that the agent is surrounded by hazards and should wait. Positions 6 through 10 direct to the nearest crate, with the tenth suggesting a bomb if the crate is adjacent to the player. Positions 11 through 15 guide toward the nearest opponent, recommending a bomb if

the opponent is within blast range. Positions 16 through 19 direct to the nearest coin, and the final position (20) is a Boolean indicating the availability and survivability of bomb placement.

During training, we noticed that the more complex states, which specify directions for all four goals, occur infrequently, slowing the learning process. To enhance performance and minimize invalid actions during gameplay, we implemented rule-based masking of the feature vector. By immediately returning the vector once a direction to safety is found, we avoid unnecessary calculations for the other three goals, reducing the search space by a factor of six. Additionally, we optimized bomb placement suggestions for the direction of an opponent: a bomb is only recommended if it is available and can be survived, further reducing the state space to a total of 335 possible states. To incorporate more information about the number of coins already discovered, we enabled the agent to count the revealed coins. Once all 9 coins have been revealed, the feature state stops the search for the nearest crate and no longer indicates the direction to it.

With this substantial reduction in dimensionality, we successfully established our initial baseline agents for various agent designs. This setup allowed us to evaluate whether each algorithm could learn efficiently. By streamlining the complexity of the state space, we not only optimized our computational resources but also enhanced the learning speed and adaptability of different algorithms under test.

Breadth-first search

To optimize our feature extraction strategies for the game environment, we employed the Breadth-first search (BFS) algorithm to efficiently compute the shortest, survivable paths to designated goals based on the proposed approach from (Kowalczyk et al. 2022). This method is particularly well-suited for our purposes due to its unweighted nature and ability to guarantee the shortest path in a graph representing the game’s layout.

BFS operates by exploring all nodes at the present depth level before moving on to nodes at the next depth level. This ensures that the first time a node is reached, it is done via the shortest possible path. For each node (representing a possible position in the game), BFS evaluates all adjacent nodes at once, expanding outward from the source node in layers. We initialize the BFS with a queue that contains the starting position, marking it as visited. The queue facilitates the orderly processing of nodes:

- Dequeue the front node.
- For each adjacent node that has not been visited: Test if the step is allowed or can be survived and if so, mark it as visited.
- Record the node’s predecessor and the direction taken to reach it.
- Enqueue the node.

This process repeats until the queue is empty or the goal is reached, allowing us to backtrack from the goal to the source to extract the sequence of moves, or directions,

that constitute the shortest survivable path. An Example of this process can be seen in Figure 4.2.

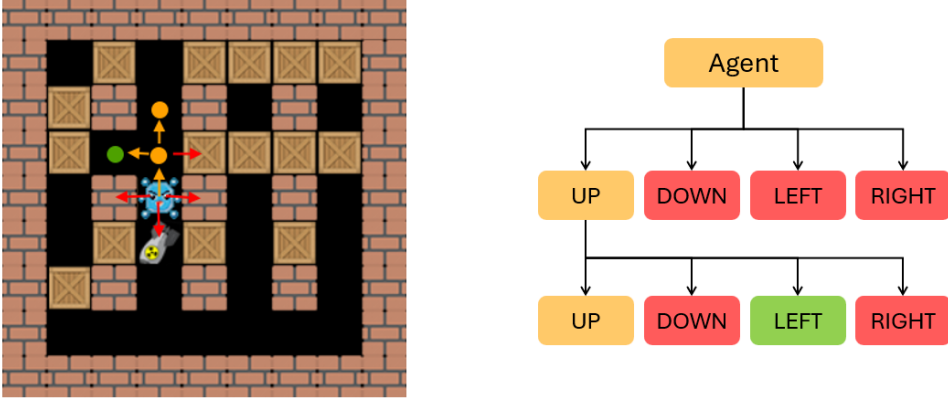


Figure 4.2: Example of Breadth-First Search (BFS) applied in the Bomberman game environment. (Left) The search is depicted on the game field, illustrating how the algorithm systematically explores positions in ascending order of their distance from the starting point. (Right) A graphical representation of the BFS process, where only viable moves are investigated. By the second layer of depth, the algorithm successfully identifies a solution.

From the calculated shortest survivable paths, the first action is extracted and encoded into the feature vector to suggest actionable moves toward each goal, such as safety locations, crates, opponents, or coins. These directions are crucial for guiding the agent’s decisions in real-time gameplay scenarios.

By integrating BFS into our feature extraction framework, we not only enhance the decision-making capabilities of our agents but also ensure that the paths recommended are the most efficient given the current game state. This methodological rigor significantly contributes to the robustness and effectiveness of our agent training protocols.

4.1.2 Complex feature extraction

In contrast to the simple feature extraction method, the complex feature extraction uses a larger, more detailed feature vector (size 29) to capture a broader range of game dynamics. The first 20 features are shared with the simple version, but the complex vector adds 5 more entries that quantify the amount of danger associated with moving in each direction. The danger is scaled between 0 (safe) and 1 (imminent death). This provides the agent with a granular understanding of potential risks when navigating the environment.

The features 24 - 26 indicate whether the agent can place a bomb and still survive, if bomb placement is possible, and if the current position is a perfect spot for a bomb. This is characterized by potentially destroying multiple crates or killing an opponent.

Two further strategic metrics are provided by feature 27 and 28. The first is the number of opponents (normalized) still alive in the game, and the second is whether the agent is currently in the lead based on score comparison with the opponents. This allows the agent to dynamically adjust its behaviour based on the competitive landscape.

This richer feature set enables the agent to make more informed decisions, balancing offense and defense while focusing on key objectives like coin collection and opponent elimination. However, it also increases the complexity of the training process, as the agent must learn to effectively utilize the additional information.

4.2 Q-learning agent

LUKE VOSS

We chose to implement a Q-learning agent as a first baseline model, motivated by its several distinct advantages. Firstly, Q-learning is a straightforward algorithm that is simple to implement and deploy, allowing us to shift our initial focus from complex algorithm configurations to developing effective feature vectors and reward structures. Additionally, as detailed in section 2.2, Q-learning converges to an exact solution rather than an approximation, enhancing the interpretability of the Q-table—it approximates rewards for chosen actions, which aids in verifying the efficacy and efficiency of learning under set parameters. Unlike model-based methods, which often involve numerous hyperparameters related to model architecture, Q-learning requires fewer hyperparameters, additionally simplifying the training process.

We implemented the Q-learning algorithm as it was introduced in our lecture, utilizing an epsilon-greedy exploration process (see section 2.2). The Q-table associates each action a with each possible feature state s , leading to a maximum matrix size of 6×335 as illustrated in Table 4.1. We employed a dictionary with states represented as tuples to manage the Q-table, leveraging Python’s average $O(1)$ time complexity for dictionary operations due to its underlying hashmap structure. The maximum value for the next state is computed based on the subsequent game state provided by the Bomberman environment, as described in section 2.2. At the end of each round, we set the max next state value to zero and update the Q-table based solely on the observed rewards. Training begins with an empty dictionary; new states are added as tuples and initialized with a reward value of zero for all actions.

We refined the epsilon-greedy exploration strategy by integrating a decay rate for the epsilon values. This adjustment allows for more extensive exploration at the early stages of training, where knowledge about the environment and action outcomes is limited. As training progresses and a substantial number of actions have been explored, the exploration rate decreases. The epsilon value for each step is determined by the following formula:

$$\epsilon = \epsilon_{\min} + (\epsilon_{\max} - \epsilon_{\min}) \cdot e^{-\lambda \cdot n} \quad (4.1)$$

where $\epsilon_{\min}$ and $\epsilon_{\max}$ represent the minimum and maximum values that epsilon can

Table 4.1: Q-Table Extract: Displays Q-values for actions across different states, illustrating the decision-making logic of the Q-learning agent.

State	UP	DOWN	LEFT	RIGHT	WAIT	BOMB
(1,0,0,0,0,0,0,0,0,0, 0,0,0,0,0,0,0,0,0,0)	0.426	-6.001	-8.357	-7.965	-2.577	-10.22
(0,0,0,0,1,0,0,0,0,0, 0,0,0,0,0,0,0,0,0,0)	0.046	-6.594	-8.083	-8.065	-1.994	-9.518
(0,0,0,0,0,0,0,0,0,1,0, 0,0,0,0,0,0,0,0,0,0)	-7.820	0.198	-9.381	-9.810	-2.000	-8.579
...	...	...	...	...	...	...

take during training, respectively, λ is the pre-set decay rate, and n denotes the current training round number.

In addition to epsilon-greedy exploration, we implemented systematic exploration. When encountering a state where an action’s value remains at the initial zero, we systematically select and test these actions. This approach, feasible due to the manageable number of states, guarantees that each action is selected at least once, addressing the challenge in epsilon-greedy methods where untested actions become increasingly unlikely to be chosen as training progresses.

4.3 Proximal Policy agent

MAX TIEDL

For our second agent we chose to implement an PPO agent due to its excellent performance in similar games (Schulman, Wolski, et al. 2017).

The PPO agent follows the standard PPO framework but is tailored for the Bomberman environment through customized features, a recurrent policy network, and GAE for learning. It utilizes an actor-critic architecture, allowing it to learn effective strategies for navigating complex game scenarios

4.3.1 Actor-Critic

The Actor-Critic architecture is a hybrid reinforcement learning framework that simultaneously learns a policy (actor) and a value function (critic). Unlike pure policy-based methods, where the agent learns only to map states to actions, or value-based methods, where it focuses solely on predicting future rewards for given states, Actor-Critic leverages both approaches.

- Actor (Policy Network): The actor is responsible for learning the policy π_θ , which maps the current state s to a probability distribution over actions. The agent selects actions by sampling from this distribution, ensuring that it maintains a balance between exploration and exploitation. In this context, the actor outputs probabilities for actions such as moving up, down, left, right, or placing a bomb.
- Critic (Value Network): The critic estimates the value function $V(s)$, which predicts the expected cumulative future rewards from the current state. This value function helps the actor assess how good or bad a particular state is, providing crucial feedback on whether the actor’s chosen actions are beneficial in the long term.

In the PPO algorithm, the critic is used to compute the advantage of a particular action, which is the difference between the observed rewards and the expected value of the state. This advantage signal is critical for guiding the actor’s policy updates.

4.3.2 Network Architecture

The PPO agent uses a Multi-Layer Perceptron (MLP) as network architecture. The architecture is designed to take as input a feature vector representing the game state and output the policy (action distribution) and the value function (state value estimate). The MLP architecture is efficient for this environment, where decisions can be made based on static game states without the need for sequential memory processing.

The structure of the networks can be observed in table 4.2. Both actor and critic network have an input layer, two fully connected hidden layers and an output layer. In case of the actor network this output layer is completed with a softmax activation to output probabilities. The Rectified Linear Unit (ReLU) activations introduce non-linearity which helps the model capture complex patterns in the game environment.

The final network structure was derived through a process of trial and error. We experimented with various network configurations, testing different numbers of layers, hidden units, and activation functions. The aim was to balance model complexity with the agent’s learning efficiency and stability.

4.3.3 Feature Extraction and Action Selection

The agent interacts with the Bomberman environment by extracting relevant features from the game state and feeding them into the neural network. The feature vector is the same our q-learning agent also uses, described in 4.1.1. This input is passed through the policy network, which outputs a probability distribution over possible actions (e.g., move up, place bomb).

During training, the agent uses stochastic action selection by sampling from the action distribution output by the network. In contrast, during evaluation, it exploits the learned policy by selecting the action with the highest probability, ensuring that it takes the most advantageous actions based on its current knowledge.

Table 4.2: Architecture of ActorCriticMLP

Component	Layer	Configuration
Critic Network		
Input Layer	Linear	Input: num_inputs, Output: 1028
	Activation	ReLU
Hidden Layer 1	Linear	Input: 1028, Output: hidden_size
	Activation	ReLU
Hidden Layer 2	Linear	Input: hidden_size, Output: hidden_size
	Activation	ReLU
Output Layer	Linear	Input: hidden_size, Output: 1
Actor Network		
Input Layer	Linear	Input: num_inputs, Output: 1028
	Activation	ReLU
Hidden Layer 1	Linear	Input: 1028, Output: hidden_size
	Activation	ReLU
Hidden Layer 2	Linear	Input: hidden_size, Output: hidden_size
	Activation	ReLU
Output Layer	Linear	Input: hidden_size, Output: num_outputs
	Activation	Softmax
Standard Deviation: Initialized as Parameter, Std: std		

4.3.4 Training Process

The agent’s training relies on the PPO algorithm, which updates the policy based on a set of experiences collected during gameplay. These experiences include the states encountered, the actions taken, the rewards received, and the values estimated by the critic network. The PPO algorithm then performs multiple epochs of updates, using mini-batches of this data to optimize the policy. Each update step aims to maximize the PPO objective from Equation 2.4, which includes a clipped surrogate loss to prevent the policy from changing too drastically between updates.

The agent uses GAE as in Equation 2.5 to compute the advantage function, which is the difference between the observed return and the estimated value of the state. GAE helps reduce variance in the advantage estimates while maintaining unbiased gradient estimates, thus improving the stability and efficiency of the learning process.

4.3.5 Mini-Batch Training and Optimization

The PPO algorithm updates the policy in mini-batches to ensure sample efficiency. The agent collects a batch of experiences and divides it into smaller mini-batches during the update phase. For each mini-batch, the policy and value function are optimized using

gradient descent.

The policy update consists of two key components:

- Actor loss: This loss is computed based on the clipped PPO objective, which ensures that the policy does not change too much between updates. The ratio of the old and new policy is used to calculate the advantage of taking specific actions.
- Critic loss: This loss is the mean squared error between the predicted value of the state and the observed return. The critic’s job is to accurately estimate the value of each state, guiding the policy towards more rewarding states.

The overall loss function is a combination of the actor loss, the critic loss, and an entropy term that encourages exploration by preventing the policy from becoming too deterministic:

$$Loss = 0.5 * CriticLoss + ActorLoss - 0.001 * Entropy \quad (4.2)$$

This loss function is minimized using the Adam optimizer, with learning rate adjustments based on the training progress.

After every few updates, the agent’s performance is evaluated based on the cumulative rewards collected during gameplay. This provides feedback on whether the policy is improving and helps track the training progress. The model is saved periodically, allowing the agent to resume training or perform evaluations using previously learned policies.

4.4 Imitation learning

MAX TIEDL

Imitation learning serves as a foundational phase in the training of our PPO agent. By learning from expert demonstrations, the agent can quickly develop a baseline policy that mimics expert behavior. This approach accelerates early training, minimizing the need for extensive exploration, and helps the agent avoid inefficient or suboptimal actions.

Although we initially also used this for the training of the q-learning agent we discovered that the agent did learn better without imitation learning.

4.4.1 Expert Dataset

The imitation learning process begins with an expert dataset that contains gameplay data from pre-recorded demonstrations. These demonstrations are generated by an expert agent or a previously trained model. The dataset includes the states, actions, rewards and masks (binary indicators that signal whether a state is terminal or not). This dataset is prepared for training by converting the data into tensors.

4.4.2 Training Procedure

The training phase involves optimizing both the actor and critic networks. The dataset is divided into training and validation sets; the training set is used to learn the policy, while the validation set monitors the model’s performance and helps prevent overfitting. During training, the actor network learns to predict a probability distribution over actions that closely matches the expert’s behavior, while the critic network estimates expected future rewards using GAE.

- Actor Loss: Computed using `CrossEntropyLoss`, comparing the predicted action probabilities to the expert’s actions.
- Critic Loss: Calculated as the Mean Squared Error (MSE) between the critic’s estimated returns and the expert’s observed returns.

The total loss, a sum of the actor and critic losses, is minimized through gradient descent. After each epoch, the model’s performance is evaluated on the validation set to adjust hyperparameters and avoid overfitting.

Once imitation learning is complete, the model is stored and used as a baseline for the subsequent reinforcement learning phase, where the agent refines its policy through interaction with the environment.

4.5 Evaluation metrics

BASTIAN MÜLLER

To judge the performance of our agents, we used a variety of evaluation metrics, building upon the metrics which are already computed by the `--save-stats` argument when simulating games with the `main.py` Python script.

4.5.1 Existing metrics

Upon completion of an entire game (call to `main.py`), the metrics are saved in a JSON file and include the following information (see Table 4.3).

4.5.2 Custom metrics

In addition to the existing metrics, we computed custom metrics to evaluate the performance of our trained agents and build interesting graphs. Specifically, we were interested in how the agent improves on aspects of the game over time.

In order to compute these custom metrics, we needed to log additional events. Fortunately, the logger of the `main.py` script already logs some of the events we were interested in. Since our training approach required multiple calls to the `main.py` script, we had to modify the logger to save the events in a different file for each call using the current date and time as file name. This way, we could read in the log files later, and compute

Table 4.3: Existing evaluation metrics

By ...	Metric	Description
Agent	bombs	The number of bombs placed over all rounds
	coins	The number of coins collected over all rounds
	crates	The number of crates destroyed over all rounds
	invalid	The number of invalid actions taken over all rounds
	moves	The number of moves made over all rounds
	rounds	The number of rounds played
	score	The total score over all rounds
	steps	The number of steps survived over all rounds
	suicides	The number of self kills over all rounds
	time	The total time taken over all rounds to determine the action
Round	round	The round number
	coins	The number of coins collected in the round
	kills	The number of kills in the round
	steps	The length of the round
	suicides	The number of suicides in the round

the custom metrics. We needed to add the location of each agent in every step to the log file.

With all desired events logged, we parsed the log files using regular expressions to match the events and compute the custom metrics.

Table 4.4 provides an overview of the custom metrics that are computed for each step of each round.

Based on these metrics, we build graphs showcasing the progress of our agents and were able to analyze single rounds as well as averages by looking at heatmaps of locations or identifying issues with the choice of actions. More on that in Chapter 6.

Table 4.4: Custom evaluation metrics per round

Metric	Description
time metrics	The time the agents took to determine their next action
action metrics	The normalized amount of actions taken by the agents
kill metrics	The amount of kills and who killed whom
suicide metrics	The amount of self kills and who committed them
location metrics	The locations of each agent and the amount of unique locations visited
coin metrics	Who collected how many coins, where they popped up and who, if anyone, collected them
bomb metrics	The amount of bombs dropped, where and who dropped them

Chapter 5

Training

In this chapter, we detail the specialized training methodologies employed for our different agents, tailored to optimize performance on low-power computational systems. Initially, we explain the sophisticated training regimen of our Q-learning agent, exploring both the underlying strategies and technical details (see Section 5.1). Following that, we transition to an in-depth examination of the training processes employed for our PPO, building further upon our Q-learning training by introducing imitation learning and further algorithmic fine-tuning (see Section 5.2).

5.1 Q-learning Agent

LUKE VOSS

In this section, we outline the efficient training methods employed for our Q-learning agent. Initially, we introduce the auxiliary rewards used and detail the structure of our reward system. We then describe the two-stage training process: pre-training (see Section 5.1.2) and main training (see Section 5.1.3). During the pre-training phase, we focused the agent on learning from immediate rewards, which laid a solid foundation without requiring extensive computational resources. In the main training phase, we expanded the agent’s capability to approximate future rewards, thereby completing the training cycle and enhancing the model’s predictive accuracy and decision-making depth.

5.1.1 Reward structure

Our initial goal was to streamline the learning process, making it both efficient and straightforward. To achieve this, we supplemented the standard rewards with auxiliary rewards (refer to Table 5.1) to ensure that the agent received immediate feedback for each action. The agent was rewarded for achieving specific objectives and penalized for actions that were either random or lacked a clear purpose.

As illustrated in Table 5.1, we rewarded running to safety with 0.2, while reaching safety earned a reward of 1. Dropping a smart bomb—defined as a bomb that the agent

Table 5.1: Auxiliary Game Rewards Configuration

Event	Reward
ESCAPING	0.2
NOT_ESCAPING	-0.2
OUT_OF_DANGER	1
CLOSER_TO_COIN	0.15
AWAY_FROM_COIN	-0.15
CLOSER_TO_CRATE	0.04
AWAY_FROM_CRATE	-0.02
WAITED_NECESSARILY	0.1
WAITED_UNNECESSARILY	-2
CLOSER_TO_PLAYERS	0.08
AWAY_FROM_PLAYERS	-0.06
SMART_BOMB_DROPPED	0.5
DUMB_BOMB_DROPPED	-8

can survive and that impacts a crate or potentially an opponent—received a reward of 0.5. Our rationale was to make the act of deploying a smart bomb appealing, especially since early training phases typically associate bomb-dropping with high risk and likely failure. High rewards for both deploying a smart bomb and escaping effectively encouraged the agent to experiment more with bomb-dropping, fostering gameplay as intended. To discourage reckless behavior, such as bomb-dropping that could lead to the agent’s death, we introduced a significant penalty. The action of dropping a suicidal bomb was tagged as “DUMB_BOMB_DROPPED” with a penalty of -8. This immediate penalty, without waiting for the bomb’s four-step detonation cycle, accelerated the learning process by enabling quicker feedback. Directional rewards such as “CLOSER_TO..” and “AWAY_FROM..” were awarded based on the agent’s alignment or deviation from set objectives. We differentiated the reward values for each goal to subtly guide the agent’s behavior according to our predefined strategies. This setup ensured that rewards were always positive when the agent took the most advantageous action, increasing the likelihood of the action being repeated. For instance, moving towards a coin always yielded a net positive reward, regardless of other potential goals pointing together in a different direction. Choosing to move towards a coin resulted in a net reward calculation of $0.15 - 0.06 - 0.02 = 0.07$, while moving towards a crate and opponent resulted in a net penalty of $-0.15 + 0.08 + 0.04 = -0.03$. By introducing our prior belief, essentially ranking *safety* > *coin* > *opponent* > *crate* as favorable goals, we admittedly constrained the training algorithm’s freedom, potentially limiting its ability to discover a globally optimal playing strategy. However, this approach was essential for rapid and effective training of our baseline Q-learning model. Moreover, our Q-learning agent’s feature vector was relatively simple and did not encompass enough game state information to identify a globally optimal strategy, given that our biases were already integrated into

the feature extraction process. Additional rewards like “WAITED_NECESSARILY” and “WAITED_UNNECESSARILY” were granted based on the WAIT action’s context. If waiting was the only viable option amidst danger, it was rewarded (0.5). Conversely, if other actions were possible, unnecessary waiting was heavily penalized (-2).

Our reward structure for the given default game rewards, detailed in Table 5.2, included a severe penalty of -10 for invalid actions. However, we adjusted this penalty and removed it from the rewards if the action could have been valid in a prior game state. This was essential, as invalid actions often resulted from situations where an opponent had priority over the same tile, unfairly penalizing the agent despite optimal decision-making based on available features. Such instances significantly hindered learning by discouraging the selection of an optimal action after severe penalties. By monitoring the frequency of penalized invalid actions during inference, we could easily assess the progression of the training process. We set the other default rewards as follows: 7 for killing an opponent, 3 for collecting a coin, 0.01 for destroying a crate, and the maximum penalty of -10 for being killed.

Table 5.2: Default Game Rewards Configuration

Event	Reward
INVALID_ACTION	-10
CRATE_DESTROYED	0.01
COIN_COLLECTED	3
KILLED_OPPONENT	7
GOT_KILLED	-10

This comprehensive reward system provided direct and immediate feedback on the suitability of actions relative to the current game state, thus fostering an effective and systematic approach to training and testing various baseline model architectures.

5.1.2 Pre-training

During pre-training, we set the learning rate to a high value of 0.9 and gamma to 0, which focused learning exclusively on immediate rewards and disregarded future rewards. This approach was effective due to the auxiliary rewards, as detailed in section 5.1.1, which ensured that each agent’s move received immediate feedback reflective of its long-term utility. This strategy facilitated a linear learning process and significantly enhanced learning efficiency. The high learning rate allowed the state values to converge rapidly to their true rewards after just a single visit, a method made viable because gamma was set to 0, focusing solely on immediate rewards.

Using the reward structure outlined earlier, we ensured that the rewards associated with each extracted game state were largely consistent, regardless of the states that preceded or followed, thereby fulfilling the Markov property. However, our proposed feature state design does not indicate which actions are feasible, leading to potential

inconsistencies when rewarding invalid actions. To mitigate this issue, we leveraged significant differences in reward magnitudes, which allowed valid actions to overwhelmingly outweigh the impact of lucky outcomes from random movements after several cycles.

For training, we designed a series of custom environments tailored to provoke specific feature states, sequentially introducing the agent to environments with different combinations of game elements:

1. Coins only,
2. Crates only,
3. Coins and opponents,
4. Crates and opponents,
5. Crates and coins
6. A thinned out version of the classic game state featuring only one opponent, only a few crates and already revealed coins.

These scenarios were deliberately structured to provoke less frequent states in typical gameplay, thereby reducing overall training time while ensuring comprehensive exposure and updating of each state. We trained the agent in each environment for 200 rounds except for the last, which ran for 1000 rounds. Further we set $\epsilon_{max} = 0.8$, $\epsilon_{min} = 0.05$ and the decay rate λ to 0.001.

To address scenarios where no bombs are available—a rarer situation in standard gameplay but still accounting for half of the potential feature states—we manually set the “Bomb Available and Survivable” feature to 0 and subjected the agent to the same environmental training cycle. After completing these two training cycles, our Q-table was fully populated with all possible states, and each action had been attempted at least once, with values mirroring the approximate immediate rewards.

Following these initial cycles, we proceeded with our main training phase, adjusting hyperparameters as needed for optimal learning outcomes.

5.1.3 Main training

For our main training phase, we utilized a reduced learning rate of 0.1. This approach ensured that the foundational results established during initial training were adjusted gradually, maintaining the integrity of the underlying model. We continued to train the agent in a classic scenario for 10,000 rounds, employing the same epsilon decay strategy as in the pre-training phase. By setting γ to 0.8, we also incrementally adjusted the Q-table to factor in future rewards in the value approximations, thus enhancing the depth and accuracy of our learning model.

This methodical approach to training, from establishing a robust foundation in the pre-training phase to refining and expanding our model’s capabilities in the main training phase, has proven crucial in developing a highly effective Q-learning agent. By carefully

adjusting the learning parameters and incorporating both immediate and future rewards, we have successfully tailored the learning environment to meet our strategic goals. The outcomes of this comprehensive training regimen have not only validated our initial hypotheses but also set a strong baseline for future enhancements and applications of our model.

5.2 Proximal Policy Agent

MAX TIEDL

This section covers the training process and hyperparameter tuning for the PPO agent implemented for the Bomberman environment. Our approach involved both imitation learning and reinforcement learning techniques, allowing the agent to learn from demonstrations while refining its policy through trial and error.

The agent uses a MLP network for the actor-critic architecture. The feature extraction pipeline ensures that each game state is transformed into a compact feature vector, which serves as input to the network. Two versions of the feature extraction were used: a smaller, basic feature vector and a larger, more detailed one.

5.2.1 Imitation Training

To improve the agent’s initial performance, we applied imitation learning which involves training the agent using pre-recorded training data. Initially we used the rule-based agent for that, but after we finished the q-learning agent we also created a data generator that is based on our best performing q-table. We used training data from 1000 rounds for both imitation models. This resulted in around 223000 data points for the rule-based data generator and around 65000 data points for the q-learning version. When using the rule based agent for imitation training the main training has to be longer, but it results in similar performing agents.

This pre-training helps the agent learn to play the first steps in the game correctly, establishing a solid foundation for further training process. It is able to bomb itself out of its corner and tries to find the first coins. But then it kills itself most of the time.

5.2.2 Main training

The primary learning phase involved training the agent in various scenarios designed to ensure robustness across a range of different situations. Each scenario was intended to isolate key aspects of gameplay, such as collecting coins, destroying crates, or competing against opponents. The training scenarios are essentially the same as with the Q-learning agent but with way less rounds. That means we have trained with coins and crates only, and different combinations also including opponents. These phases had each between 20 and 50 rounds followed by a final classic scenario with 3 rule based opponents that lasted 200 rounds. For the last phase the learning rate was also decreased, ensuring that

the previous phases are not neglected. We also trained another 100 rounds with 3 of our q-learning agents and a lower learning rate, but the results were not very pleasing.

In the end we discovered that eliminating the phase-based training in favor of training exclusively on the classic scenario lead to better results. For this approach, we reduced the discount factor γ . This also improved the agents' ability to perform well across a wide range of game scenarios, including those not explicitly trained. After extensive training with different hyperparameters it showed that with around 200 rounds the agent is at its best performance. Further training did not lead to significantly better results or even made it worse.

5.2.3 Hyperparameter Tuning

We experimented with several hyperparameters to optimize the performance of the PPO agent. Key parameters included the learning rate, the discount factor γ , the GAE parameter τ , and the clipping range ϵ . Through systematic exploration, we determined that:

- Learning rate: A lower learning rate of 10^{-4} helped stabilize training, preventing the agent from making large, unstable updates to the policy.
- Discount factor (γ): Initially, a value of 0.99 was used to encourage the agent to prioritize long-term rewards. However, through experimentation, we found that a much lower value of 0.3 was more effective, reducing the influence of future rewards and allowing the agent to focus more on immediate outcomes.
- GAE parameter (τ): A value of 0.95 was found to offer a good trade-off between bias and variance in the advantage estimates.
- Clipping range (ϵ): We experimented with different clipping values between 0.1 – 0.3, with 0.2 providing the best results by allowing small updates while avoiding drastic policy changes.

Chapter 6

Experiments and Results

This chapter focuses on our experiments and their results. It will start with a brief overview of the training process but mostly referring to the detailed description given in Chapter 5. The chapter will present the results of the training process and the following testing of the agents using some selected metrics described in Section 4.5 to keep the Chapter at a reasonable length. In a second step, the results will be compared to the rule-based agents and discussed in detail.

6.1 Training results

6.1.1 Training process of the Q-learning agent

BASTIAN MÜLLER

The training process of the Q-learning agent is described in detail in Chapter 5.1. The overall training from start to finish resulted in over 18,000 rounds played over a range of scenarios. The length of the logs for a single round lasting 400 steps is around 3,700 lines. This results in a total of around 60,000,000 lines of logs. The training process took around 2.6 seconds per round on a single core of an AMD Ryzen Threadripper 2950X at 4.4 GHz. However, the large size of the log files caused delays in I/O operations, resulting in a significant decrease in performance.

The following graphs show the performance of the agent over the course of the training.

Figure 6.1 shows the amount of rounds per training or test step. The first 12 steps are the Pretraining steps, played for 200 rounds each. Except for the 6th scenario, which was played for a thousand round because it still filled new table entries (so it experienced new feature vector configurations) well beyond the 200 round mark. The main training took place over 15x1000 rounds, while the evaluation of the agents after an additional 1000 rounds of main training took 14x250 rounds, except for the final agent, which we chose to test more extensively.

Gaining insights by observing the pretesting and main testing rounds proved difficult for different reasons. During pretraining, the agent did not have a bomb available for

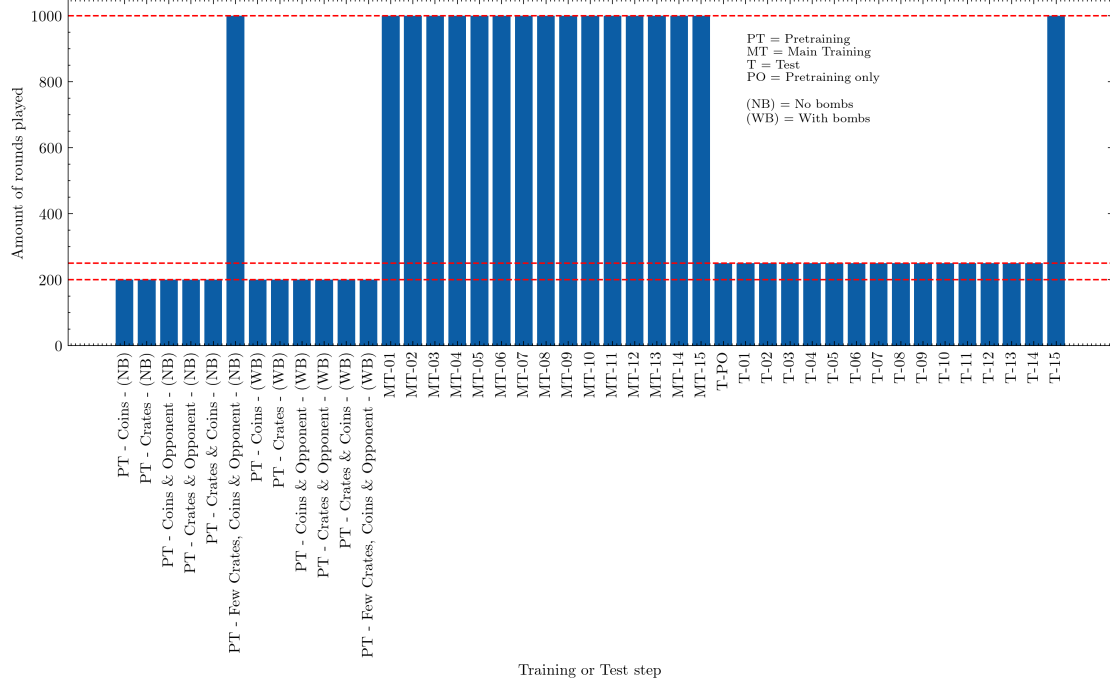


Figure 6.1: Amount of rounds per training or test step: The PT steps show the 12 pretraining steps in their different scenarios, with or without a bomb. The MT steps show the 15 main training steps, while the T steps show the evaluation of the agent after the training.

the first 6 steps and played mostly alone in highly customized environments, rendering it useless when matched against the rule-based agent. Another reason has to do with the amount of logs we wanted to collect. To keep the log files from not growing larger than 500 MB per run, we chose to set the amount of rounds played to 1000 and to run the training 15 times in succession using the model weights from the previous run in the next. This ensured a continuous training, but due to our high choice of $\epsilon_{\max}$ of 0.8 the agent's performance at the beginning of each round was pretty bad, while it improved during the course of the 1000 rounds played in each iteration. This made it hard to compare the get a good insight into the agent's performance from looking at the training rounds. A strong indicator for the bad training performance is the amount of collected coins per round, displayed in Figure 6.2. During main training, the agent used too many random actions to get a good performance, but still made progress. We chose collected coins as metric instead of the score or kills, because in the beginning of the pretraining the agent is only able to collect coins and since a kill is worth 5 points, we then couldn't compare the agent's performance over all rounds. Since the performance after more than 10,000 rounds decreased again, we chose to use this agent as our final Q-learning agent and stopped our official training process here.

For these reasons, we chose to focus our attention on the test rounds (labeled T-PT

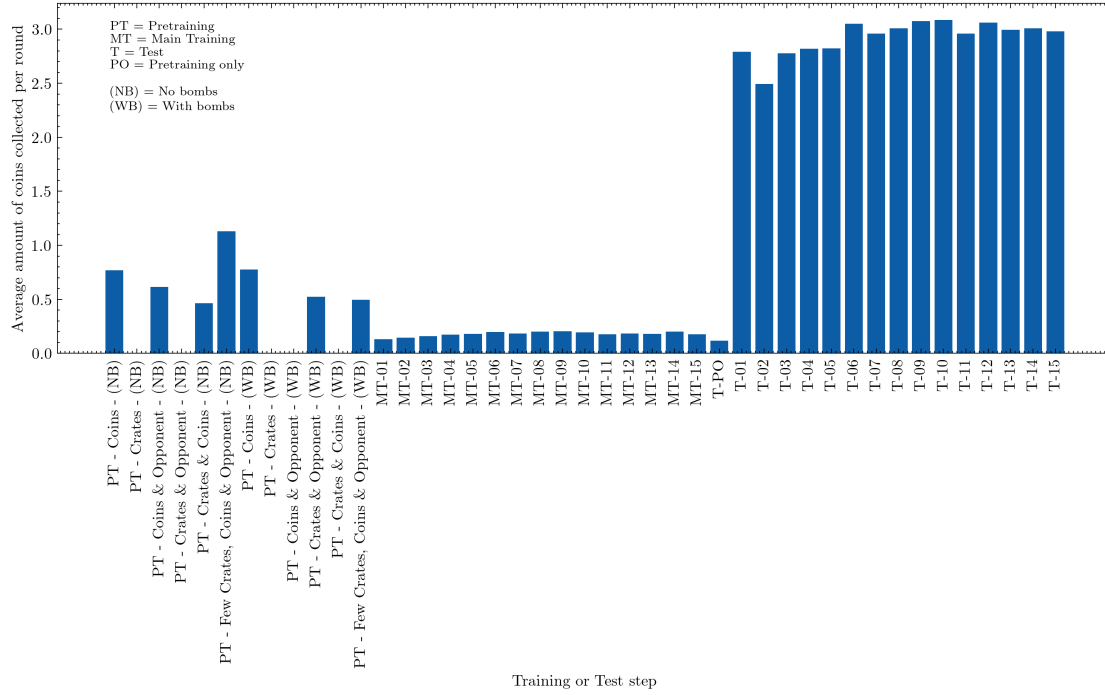


Figure 6.2: Amount of coins collected per training or test step: See Figure 6.1 for more information.

to T-15 in Figures 6.1 and 6.2), where we played 250 rounds¹ in the classic scenario against three rule-based agents. The score after each step of main training can be seen in Figure 6.3.

While we could present our findings for themselves in this part of the report, we believe that they are especially interesting when presented in the context of the actions of the rule-based agents as baseline comparison or against the PPO agent in Section 6.2. Furthermore, our agent demonstrated rapid learning after the initial pretraining, similar to the PPO agent discussed in the subsequent section. As a result, conducting multiple analyses would be redundant in terms of content.

6.1.2 Results for the PPO agent training

MAX TIEDL

The training of the PPO agent was divided into two key phases: pretraining using imitation learning, followed by a main training phase. This was also elaborated in chapter 5.2. For imitation learning, 10,000 data points were generated by a custom data generator based on a previously trained Q-learning agent. This allowed the agent to learn basic behaviors before the reinforcement learning phase, reducing the amount

¹1000 for T-15, as stated before

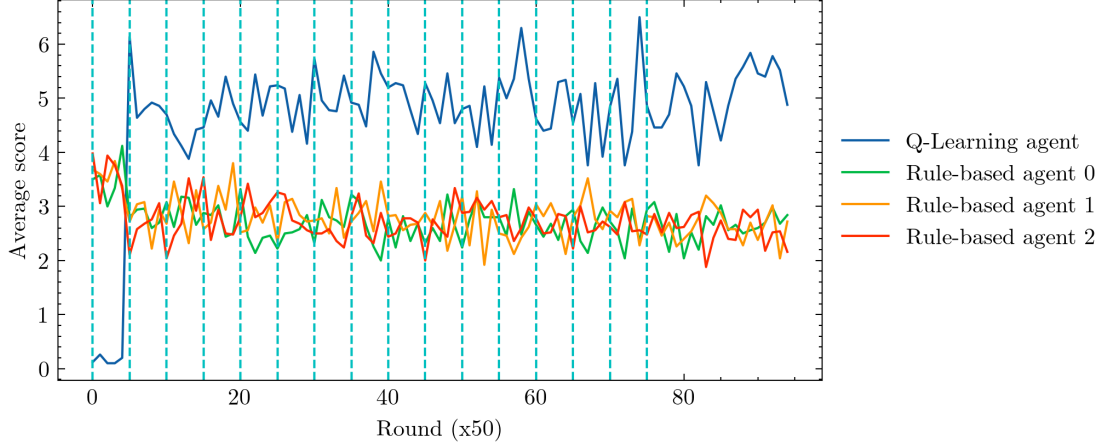


Figure 6.3: Average score of the Q-learning agent against three rule-based agents: The vertical lines indicate a new model. The first model is the pretrained model, while the last model is the final model that was tested for 1000 rounds. To increase the readability of the graph, the average over 50 rounds is displayed instead of the exact score for each round.

of random exploration early on. We also tested with more data points, but it did not improve the performance of the agent.

The main training phase spanned 200 rounds, with performance evaluated every few rounds by playing 100 matches against three rule-based agents.

As shown in Figure 6.4a, the agent’s average score per round increased rapidly within the first 10 rounds, jumping from about 1 to 4 points per round. This rapid learning phase was followed by more gradual improvements, with the agent stabilizing between 4.5 and 6 points per round towards the end of training, reflecting the agent’s policy refinement over time. The same progress can also be seen for the other metrics including kills, bombs and coins.

Figure 6.4b shows the agent’s ability to reduce suicides throughout training. Initially, the average suicides per round were close to 1, but this figure dropped quickly to around 0.5 after 10 rounds. As training progressed, the agent further reduced suicides to approximately 0.2 per round, suggesting it became increasingly adept at navigating the environment without self-destructive actions.

The following table shows also the other game metrics with their value at the beginning of the training and what value it has settled on after 200 rounds. Bombs, coins, kills and crates have the same curve as the score in figure 6.4a but with different values. The trend of the time and the invalid moves are comparable with figure 6.4b so there graphs are also omitted.

Table 6.1: Other Game Metrics

Metric	Imitation agent²	After 200 rounds²
Bombs	6	33
Coins	0.5	3
Kills	0	0.4
Destroyed Crates	15	30
Invalid moves	1	0.15
Time	0.25 s	1.3 s

Overall, the data indicates that the PPO agent rapidly learned effective strategies in the early stages of training and continued to fine-tune its balance between scoring points and avoiding risky behaviors, leading to a stable and effective policy by the end of the training.

As previously stated there are two versions of the feature vector and although the large feature vector provides richer context, it did not perform significantly better during training compared to the smaller version, possibly due to the agent’s difficulty in fully exploiting this detailed information. The smaller one is the same used by our q-learning agent. Only for the imitation learning based on the rule-based agent it made a significant difference, however after the main training phase it showed to be worse to rely on the large feature vector.

6.2 Performance comparison

BASTIAN MÜLLER

In this section, we compare the performance of the trained agents to the rule-based agents as well as let both of our models compete. We will use selected metrics described in Section 4.5 to evaluate the performance of the agents.

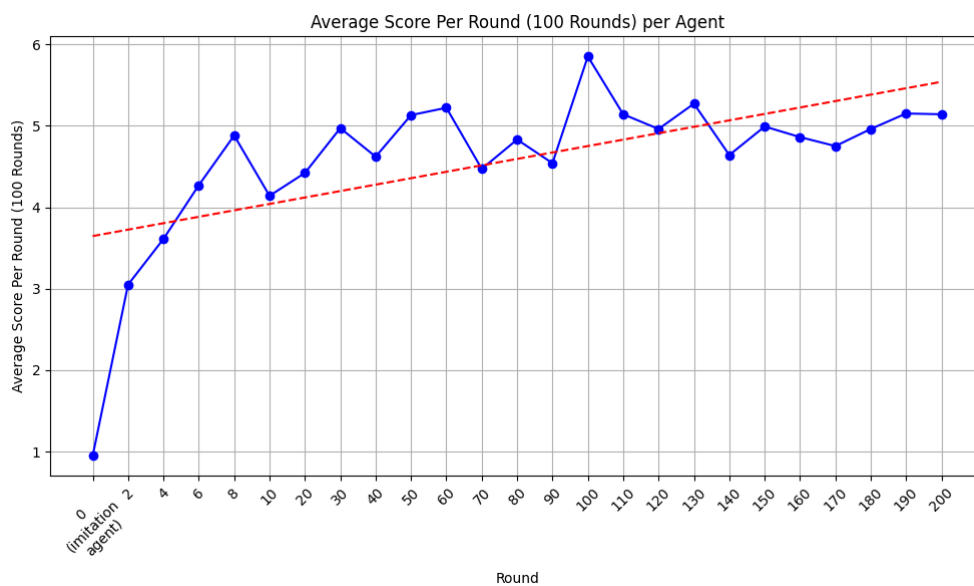
6.2.1 Q-learning agent

To start off, both of our agents easily stay within the limits of the runtime requirement of 0.5 seconds per step. The Q-learning agent takes around 10 times as long as the rule-based agents, while the PPO agent takes around 12 times as long.

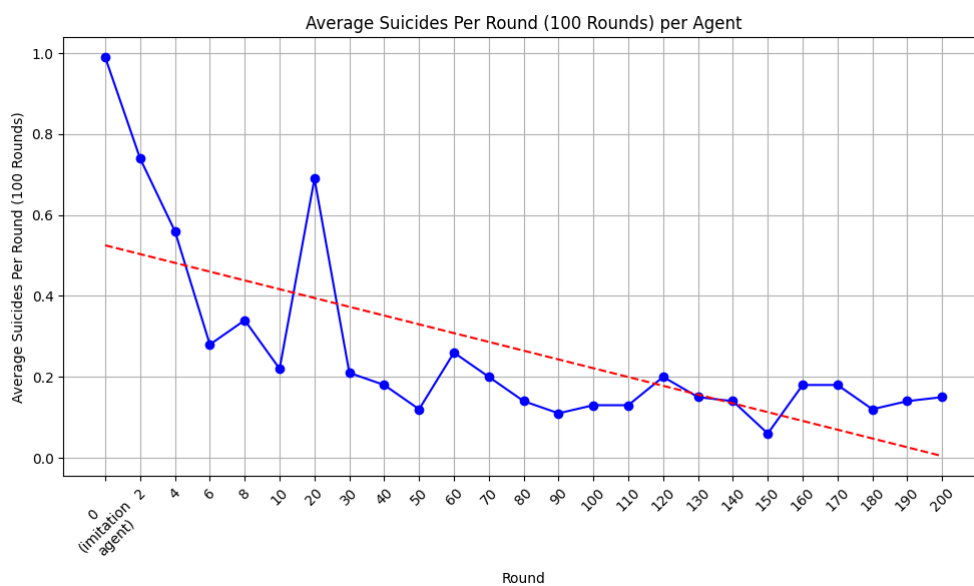
To learn more about the agents movements in during the game, we plotted the chosen actions of the rule-based agents and our Q-learning agent in Figure 6.5. Please note that if not specified otherwise, the following plots are generated using data collected with our final Q-learning agent.

Figure 6.5 shows an interesting perspective. Specifically, it shows our agent moving significantly more down and up instead of right or left. This behavior might seem strange on first sight, but makes sense when considering how the agent searches for available

²(average per round)



(a) Average Score



(b) Average Suicides

Figure 6.4: Training progress of the PPO agent

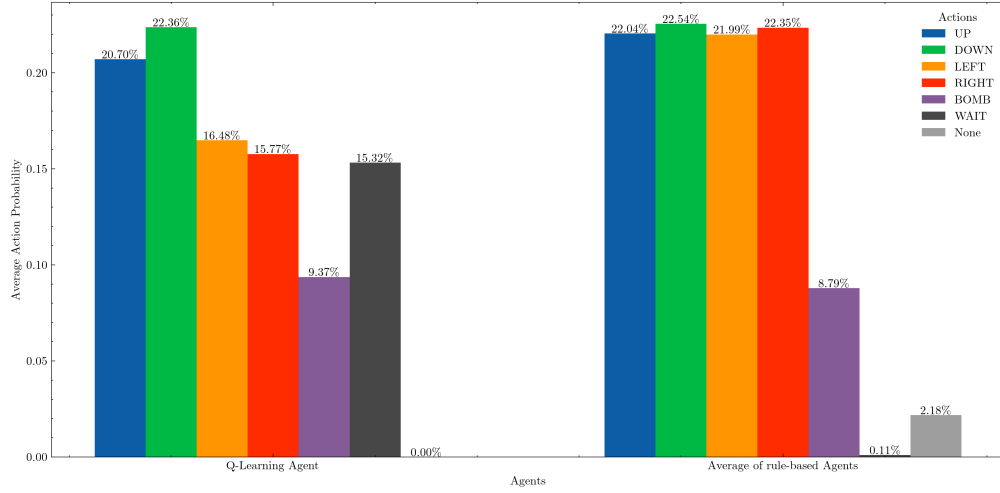


Figure 6.5: Comparison of actions chosen by the Q-learning agent and the rule-based agents (average of three agents) in the classic scenario.

moves using BFS. The agent first checks if it can perform a move in the directions UP or DOWN, before considering LEFT or RIGHT. This is due to the way the agent’s feature vector is constructed, which is based on the agent’s current position and other information (see Figure 4.1). We decided to investigate further and look at a heatmap of the agent movements. Figure 6.6 clearly shows that each agent is mostly in its own corner but also that the Q-Learning agent explores the map specifically more downwards than to the right of the field. This lead to the assumption that our agent also stays more within the fields it already explored. That turned out to be the case with the average amount of unique locations visited per survived step by our agent being around 0.24, while the rule-based agents visit 0.28 unique locations. Due to the high sample size of 1000 rounds, this difference is significant.

Despite our agent not using the entire map to its advantage, it manages to pull off a good performance (see again Figure 6.3). While the rule-based agents take around 2.9 steps to collect a coin our agent manages to collect it in around 2.7 steps.

6.2.2 PPO agent

The PPO agent performs better but very similarly to the Q-Learning agent, so we chose to omit an entire repetitive section on its performance.

6.2.3 PPO vs. Q-learning

While the simpler Q-Learning agent performed well, it was no match for the PPO agent. Over 1000 rounds against three rule-based agents, it got an average score of 5.2, while the Q-Learning got 5.04 on average. In a direct comparison over 100 rounds, the Q-Learning agent got an average score of 4.2, while the PPO agent managed an average

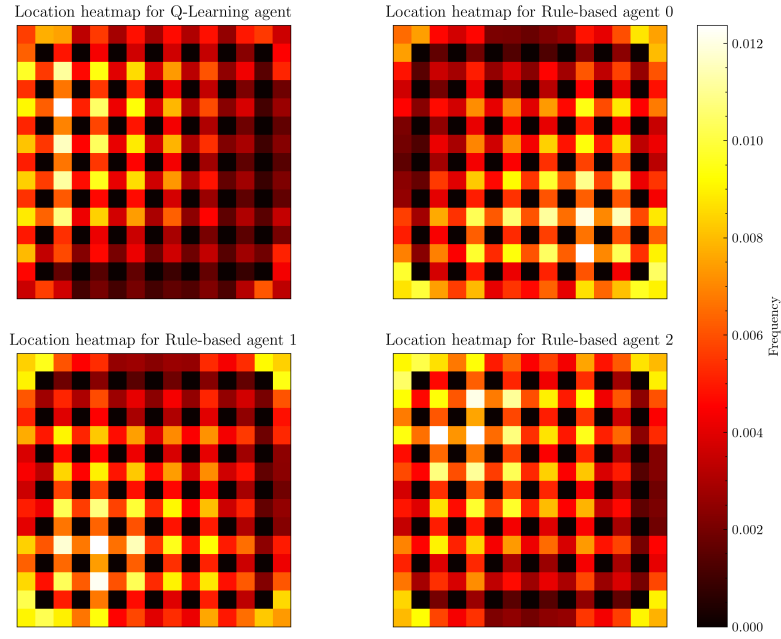


Figure 6.6: Heatmap with frequencies for each field for all agents

score of 4.3 per round.

Figure 6.7 shows the amount of suicides and kills for each agent over 250 rounds in a matrix. While the PPO agent has a higher amount of suicides, it also has a higher amount of kills. The Q-learning agent survives longer but does not kill as many enemies. Since surviving until the end of the round does not give any points, the PPO agent comes out on top.

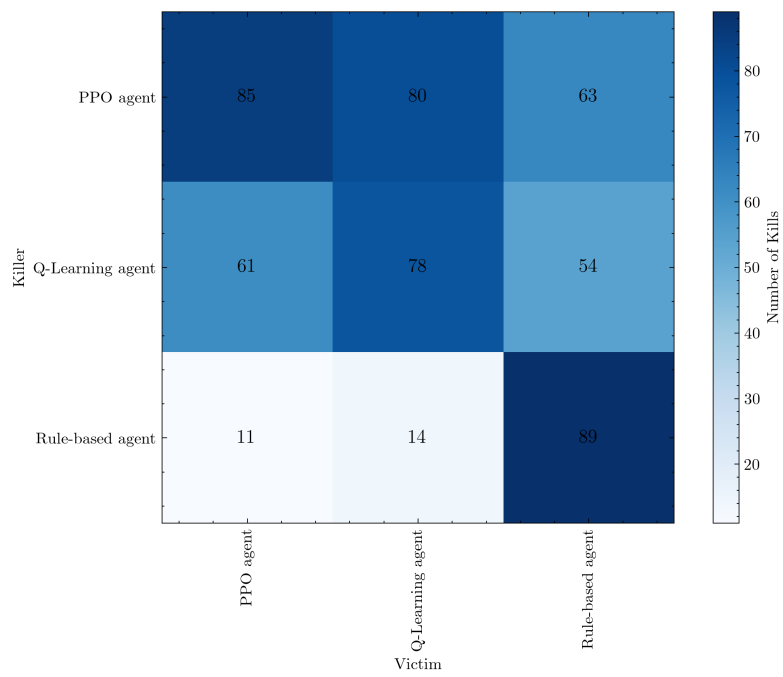


Figure 6.7: Confusion matrix indicating which agent killed or suicided how many times over 250 rounds in the classic scenario.

Chapter 7

Conclusion and Outlook

BASTIAN MÜLLER

In this Chapter we reflect on the methods and results of our agent, and speak to future work.

7.1 Conclusion

First, we achieved our goals of developing an agent that not only consistently beats the rule-based-agents but also scores more than 5 points per round. And we did it twice. Our simpler Q-Learning approach was able to score an average of 5.04 over 1000 rounds against 3 rule-based-agents, while the PPO-based model even outscored this one by a significant margin.

Second, our strategy of first focussing on building a simple and effective feature vector, and then going with an easy to implement method like Q-Learning, before turning to more sophisticated methods paid off. This choice of strategy allowed us to experiment with different models while having a competitive agent ready to go, if we fail.

Overall, it seems that we picked the correct strategy.

7.2 Outlook

This section will give an outlook on possible approaches, we did not have time to implement or test. It will also give an outlook on possible improvements to the current agents as well as the environment.

7.2.1 Future Work

An approach that we would have liked to explore is Deep Q-Learning. This method is a combination of Q-Learning and Deep Learning. It has been shown to be very successful in many environments, especially in games. We believe that this method could have been successful in our environment as well. Another approach that we would have liked

to explore is the use of more advanced models like Long Short-Term Memory (LSTM) or Gated recurrent unit (GRU). These models are especially useful when dealing with sequences of data. As our environment is a sequence of states, we believe that these models could have been successful here. However, we did not have the time to implement and test these models.

7.2.2 Improvements

We believe that there are several ways to enhance our agents. One significant improvement could be the adoption of a new feature vector. Although our current feature vector provided sufficient data for our agents to achieve good results, we speculate that a larger or more sophisticated feature space could have further improved their performance. In this regard, we would have liked to explore the utilization of Convolutional Neural Networks (CNNs) to automatically extract features from the game state. CNNs have demonstrated great success in image recognition tasks, and we believe they could have been equally successful in our environment. Given the size of the feature space, this approach would have been particularly beneficial for the PPO agent.

Bibliography

- Bai, Yuntao et al. (Apr. 12, 2022). *Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback*. DOI: 10.48550/arXiv.2204.05862. arXiv: 2204.05862 [cs]. URL: <http://arxiv.org/abs/2204.05862> (visited on 08/20/2024). Pre-published.
- Beysolow II, Taweh (2019). “Reinforcement Learning Algorithms: Q Learning and Its Variants”. In: *Applied Reinforcement Learning with Python: With OpenAI Gym, Tensorflow, and Keras*. Ed. by Taweh Beysolow II. Berkeley, CA: Apress, pp. 55–76. ISBN: 978-1-4842-5127-0. DOI: 10.1007/978-1-4842-5127-0_3. URL: https://doi.org/10.1007/978-1-4842-5127-0_3 (visited on 09/02/2024).
- Géron, A. (2019). *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O’Reilly Media, Incorporated. ISBN: 9781492032649. URL: <https://books.google.de/books?id=OCS1twEACAAJ>.
- Grondman, Ivo et al. (Nov. 2012). “A Survey of Actor-Critic Reinforcement Learning: Standard and Natural Policy Gradients”. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 42.6, pp. 1291–1307. ISSN: 1558-2442. DOI: 10.1109/TSMCC.2012.2218595. URL: <https://ieeexplore.ieee.org/abstract/document/6392457> (visited on 09/02/2024).
- Kowalczyk, Dominik et al. (Mar. 17, 2022). *Developing a Successful Bomberman Agent*. URL: <https://arxiv.org/abs/2203.09608> (visited on 09/02/2024). Pre-published.
- Meisheri, Hardik et al. (2019). *Accelerating Training in Pommerman with Imitation and Reinforcement Learning*. arXiv: 1911.04947 [cs.LG]. URL: <https://arxiv.org/abs/1911.04947>.
- Mnih, Volodymyr et al. (2013). *Playing Atari with Deep Reinforcement Learning*. arXiv: 1312.5602 [cs.LG]. URL: <https://arxiv.org/abs/1312.5602>.
- Moerland, Thomas M. et al. (Jan. 3, 2023). “Model-Based Reinforcement Learning: A Survey”. In: *Foundations and Trends® in Machine Learning* 16.1, pp. 1–118. ISSN: 1935-8237, 1935-8245. DOI: 10.1561/22000000086. URL: <https://www.nowpublishers.com/article/Details/MAL-086> (visited on 09/02/2024).
- Resnick, Cinjon et al. (2022). *Pommerman: A Multi-Agent Playground*. arXiv: 1809.07124 [cs.MA]. URL: <https://arxiv.org/abs/1809.07124>.
- Rummery, Gavin Adrian and M. Niranjan (1994). “On-Line Q-learning Using Connectionist Systems”. In: URL: <https://www.semanticscholar.org/paper/>

- On-line-Q-learning-using-connectionist-systems-Rummery-Niranjana/7a09464f26e18a25a948baaa736270bfb84b5e12 (visited on 08/20/2024).
- Sallab, Ahmad El et al. (Jan. 29, 2017). “Deep Reinforcement Learning Framework for Autonomous Driving”. In: *Electronic Imaging* 29.19, pp. 70–76. ISSN: 2470-1173. DOI: 10.2352/ISSN.2470-1173.2017.19.AVM-023. arXiv: 1704.02532 [cs, stat]. URL: <http://arxiv.org/abs/1704.02532> (visited on 08/20/2024).
- Schulman, John, Philipp Moritz, et al. (2018). *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. arXiv: 1506.02438 [cs.LG]. URL: <https://arxiv.org/abs/1506.02438>.
- Schulman, John, Filip Wolski, et al. (2017). *Proximal Policy Optimization Algorithms*. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- Silver, David et al. (Dec. 5, 2017). *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. DOI: 10.48550/arXiv.1712.01815. arXiv: 1712.01815 [cs]. URL: <http://arxiv.org/abs/1712.01815> (visited on 08/20/2024). Pre-published.
- Sutton, Richard S. (Aug. 1, 1988). “Learning to Predict by the Methods of Temporal Differences”. In: *Machine Learning* 3.1, pp. 9–44. ISSN: 1573-0565. DOI: 10.1007/BF00115009. URL: <https://doi.org/10.1007/BF00115009> (visited on 08/21/2024).
- Sutton, Richard S. and Andrew Barto (2020). *Reinforcement Learning: An Introduction*. Second edition. Adaptive Computation and Machine Learning. Cambridge, Massachusetts London, England: The MIT Press. 526 pp. ISBN: 978-0-262-03924-6.
- Vaswani, Ashish et al. (2023). *Attention Is All You Need*. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- Watkins, Christopher (Jan. 1, 1989). “Learning From Delayed Rewards”. PhD thesis. England: University of Cambridge.
- Watkins, Christopher J. C. H. and Peter Dayan (May 1, 1992). “Q-Learning”. In: *Machine Learning* 8.3, pp. 279–292. ISSN: 1573-0565. DOI: 10.1007/BF00992698. URL: <https://doi.org/10.1007/BF00992698> (visited on 08/20/2024).